

Enhanced Data Security in Network Environment

Ram Krishna Akuli
Scholar CVRU, Bilaspur,

Dr. J. Durga Prasad Rao
Additional Director and Head,
Computer Science Department
Shri Shankaracharya Mahavidyalaya
Junwani, Bhilai

Dr. Satyendra Kurariya
Head, Computer Science Department
Mata Gujari College
Jabalpur

Abstract - This study is based on the development of a new secure protocol for remote calls. The secure protocol design specification and descriptions are analysed comparing them with the existing protocols. The protocol is designed in a simple way with in built security features. Cryptographic modules can be exchanged due to the flexibility of the new approach depending on various issues and security matters. The developed protocol in this study is platform independent. The security levels of the new secure protocol are properly analysed with desired results. Comparisons with other existing technologies like CORBA or the RMI were also addressed. The results show that creation of a secure network protocol universally acceptable. Although all the bugs and security issues were not addressed as they keep evolving on a daily basis.

Keywords: - *Cryptographic Protocol, Secure Remote Protocol, Network Security*

I. INTRODUCTION

In the modern era that we are living in secure programming, systems have become more and more important. This is as a result of the reliance of computer programs in almost every aspect of our daily activities [1,9,15]. Various achievements have been made to secure programming. For instance, a while ago it was not an easy task to create a connection by use of network and make use of it in programming. The very first network software's, utilized network datagrams and network sockets. These programs were complicated, regular cases of damaged packets and the loss of connection. They were also costly and needed binary data conversion and representation that the programmer would create. Therefore, the need for a cheaper single program that was divided into independent and few sections run and controlled by separate hosts. The available technologies that serve this purpose are RMI, RPC and also CORBA [3,5,17]. These technologies have required engineers to create a program that is similar to that of a single machine. They are less costly for bigger projects and the responsibility for the network layer is transferred from software engine to the network library purveyor.

CORBA, RPC, and RMI technologies are classified as binary protocols that require the programmer to use suitable libraries [7,16,24]. Just like SOAP and REST, they fall in the text-based remote technologies. It should also be noted that these technologies are different in some way. All of them make network communication easier and simple. They permit the programmer to divide the software into sections that are operating at the same time on separate hosts within the network.

Unfortunately, all these technologies have security concerns. Not even one of them is security oriented. It is possible to hack all these remote protocols which can be dangerous in different situations. The security mechanisms that are applied in these remote protocols are not safe for example, SSL, SSH, Kerberos and many others. The vulnerabilities and weak authentication in the process of authorization cannot rely on cryptographic algorithms alone. In the case of software libraries that were established with programming guidelines that were secure are also still not safe. Attackers can make good use of security bugs such as incorrect permission, poor authentication, buffer overflows and many others [7,13,14]. This created the need to create a network protocol that possesses security mechanisms that deal appropriately with the current requirements. Therefore, this study shows that there is potential for creating design, various specification and implementation of a universal, new network protocols intended for remote function, procedures or even methods that is characterized by:

- a. Security mechanisms that are build-in
- b. The security mechanisms will be suited in a manner that aligns the security level to the demand of various systems utilizing the protocol. This will involve even the legal requirements of the client.
- c. The new protocol will have no operating system dependencies and hardware.

The Secure Remote Protocol (SRP) addressed in this study fits the characteristics mentioned above.

II. SRP SPECIFICATIONS

Level of architecture in SRP

The aim of this study is to come up with a simple and easy protocol, basing itself on simple architecture. The Secure Remote Protocol (SRP) is designed to be independent of the programming language that is used and system of architecture [2,18,28]. It comprises of some stubs that are allocated to various languages and platforms. It is from the secure interface definition language that the software developer should produce the source code and instruct the functions existing in produced code [8,21,26]. The produced codes should utilize stubs to start the remote methods. This applies to other remote protocols like COBRA, RMI or RCPC [1,4,25,27]. It should also be noted that in SRP stubs are in control of authorization, communication, and network communication.

The programmer must be aware of the signature of the used remote methods. He is not required to be aware of the IP address or port. The main registry, stores port, IP address and other data are associated with the method. The local cache is added to the whole architecture. Cryptographic modules control decryption, encryption, together with hash functions that are responsible for data integrity and confidentiality [6,7,22,23]. This means that developers were not required to carry the cryptography about [37]. Another important component that is available in the SRP is the 'Authentication and Authorization Manager' better known as the AA manager. Authenticated users were obliged to conduct all the revocations. Furthermore, each and every remote action ought to be permitted. Developers are also allowed to implement the AA Manager. It is very simple architecture as programmers produce sources from files of

interface description and later form their program [34,35,36]. The other components were conveyed by the developer of the SRP library.

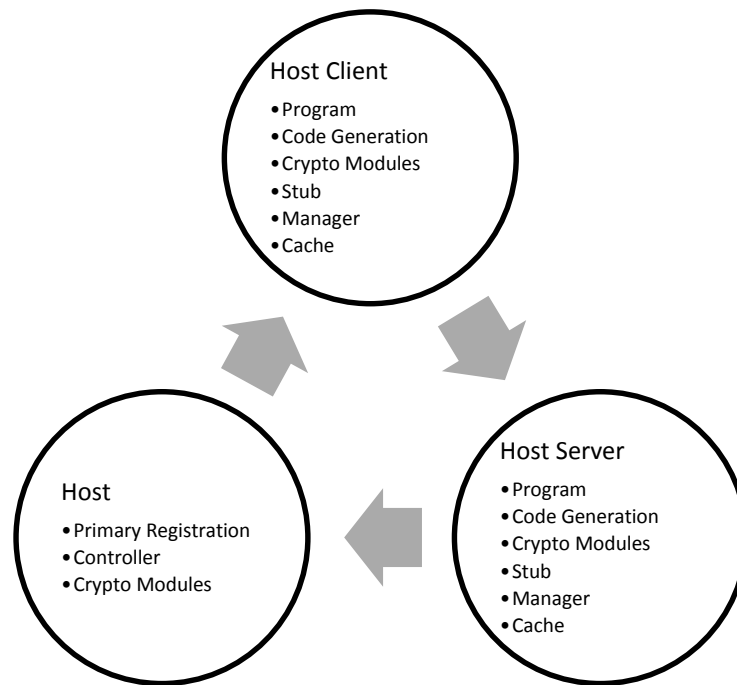


Figure 1: Simple architecture of SRP

III. SRP DATA REPRESENTATION

Since the protocol is so much technical and detailed, data representation ought to be appropriately defined. Simple types and basic data were sent to the big endian representation, where they are converted by stubs into local architecture if they are not compatible with the architecture available [38,39]. The whole process was carried out transparently. Each type was numbered with a unique identification number to differentiate it from the rest. The identification numbers are usually required by the protocols and are useful in data representation [3,11,12,29]. Data types that were used are Boolean, string 8, string 16, int16, int64, int32, int8, string 24 and many more.

Network

The frames of the network that were sent via the network connecting to hosts contained two sections of the frame. The first section was a header, and the other was the encrypted part. [10,31,32] In the SRP, the client had the mandate to keep the encrypted part confidential or he or she could share it.

IV. SECURE INTERFACE DESCRIPTION LANGUAGE (SIDL)

Secure Interface Description Language (SIDL) also forms the specifications of SRP the language bases itself on XML. Each of the specifications that form the SIDL is named to be identified by the code generator. The names given are useful in file names and classes. Every function(C), methods (the language of the object) and remote procedure (Pascal) are referred as function. All the functions had different returned type and name [33,34,35]. All the parameter had two mandatory XML parameters: type and name. The produced codes had a total of three parts: client, server and common. The common part was supposed to be used by both the server and the client. It controlled the process of registration of functions that was performed in the Main Registry. In the part client functions that were ready to be used were present. In the SRP, the remote method implementation is the duty of the programmer.

SRP was divided into Java sections such as `utils.crypt`, `connectors`, `utils.packets`, `utils.types` generated code or components. In each divided section, there were groups of the same class. For instance, the `utils` type was responsible for handling all the SRP types, the `connectors` had classes that controlled connections involving classes from various components like the Stub or Main Registry. Every connection was responsible for any exception. These exceptions were like receiving inaccurate data, tracking lost connections or losing connection [40,41]. The mechanisms of Cryptographic in SRP are build-in in the `utils.crypt` packages adapters [2,19,30]. The classes offer support to appropriate algorithms. The developer of the software, in this case, is not required to carry out the process of proper implementation. In the issue of authentication and authorization, the software engineer was given the mandate to control it. The default mechanisms also are build-in although if changes are necessary, they were allowed.

SIDL code generator was written to form a complete implementation of the SRP. This part was not required by the user but was meant for the software developer to produce stubs from the files that had interface description in their SIDL. In this case, the parser language is independent and universal. The output, on the other hand, was Java code. However, changes can be done at any time.

Quality of the code

To come up with a secure programming network as intended, the codes were protected by a series of test cases. These tests were: integration tests, unit tests, sanity tests and component tests cases [3,42,43]. Network scanners were never utilized because there was an absence of attack vectors linking with SRP. All the bugs that we are all familiar with were fixed in place as implementation was being carried out. The source codes were inspected, and the necessary security issues were addressed appropriately. The find bugs were able to locate the vulnerabilities. It was also realized that the find bugs software could not function if the 'white character' were present in the route leading to the executable file.

V. IMPLEMENTATION OF SRP

Cryptography-related stuffs

Cryptography was necessary because clear-text protocols are never secure because they can be breached or faked at any time. Cryptographic algorithms provide confidentiality messages over the network that is sensitive and classified. Additionally, personal data is similarly protected. Mechanisms of data integrity also were implemented by use of the one-way hash functions. Cryptographic mechanisms also provided the protection from replay attacks with build-in mechanisms into the protocols [44,45,46]. Due to the dangerous state of the replay attacks, the random digits that are usually used to protect protocols were randomized in the most complex way. Software that is secure with the quality random generator was put in place. In general, the SRP fulfilled all the requirements of cryptography mechanisms.

Authentication, Authorization, and Accounting

The process of authentication involved identifying and verifying the user who wishes to access or use the device. The whole process creates a trustful relation between the application and the user. The authentication procedure depended on the type because there are many types of authentication available. The SRP provided a mutual authentication mechanism that is the safest way of authentication [47,48]. The both sides involved had to authenticate each other. There were few types of authentication mechanism. For instance, the first type is involved pin codes or passwords. The passwords rules stated that they are not to be shared, as they are used as identities of the users. It is advisable that we use at least two types of authentication in our networks to protect it efficiently. The reason behind the use of more authentications is to make it hard for intruders to fake or breach them.

Since authentication is not sufficient alone, authorization was also used. This is the processes of investigating the user's rights. This gave the need for proper user management and appropriate configuration in the SRP. The privileges of the users were limited to prevent them from conducting unimportant actions. Therefore, rule-based access control (RBAC), Mandatory access control (MAC), role-based access control (RBAC) and discretionary access control (DAC) were all put in place [2,49, 50]. In SRP, the software designer is allowed to use authorization that is a default. The designer can also be in full control and offer authorization to other parts depending on the system requirement.

On the other hand, accounting was necessary, which is the presence of logs originating from operations that have been performed. Only authorized and authenticated users were allowed to carry out accounting.

Denial-of-service

To offer full protection, the SRP had to be able to be secure from Denial-of-service (Dos) attacks. To come up with such high-quality protection, the protocol had to possess limits for specific operations [51,52,53]. The administrator was responsible for such limits and controlled them. The origin of the attacks was also supposed to be blocked. The issue of attack was left open in the implementation of the SRP.

Input validation

Most threats occur as a result of input validation that is insufficient. To secure the SRP, the input specified by the protocol description should be validated appropriately. Additionally, every program was supposed to validate each input despite the fact that the protocol implementation is protected. SRP, which is implemented in Java, deals with input validation as expected [2]. Race conditions issues were also addressed during the designing period, with the protocol serializing all the network requests.

Third-party software and unsafe functions

It is sometimes hard to determine whether the third-party software's that were utilized in the implementation of protocols are free from vulnerabilities and bugs [54,55,56]. Therefore, the software licenses and legal issues were well addressed to prevent risks that end-users and developers take. The SRP licenses were present and valid and were utilized with Java Runtime Environment licenses.

Static Analysis Tools

All the warnings were addressed and fixed in the secure implementation. Static analysis tools like Klocwork and Yasca were utilized [66,67]. These tools diagnose problems that are related to security such as race conditions, memory leaks, unsafe functions usage, insufficient input validations and many more issues. The SRP took such security issues into consideration and were fixed appropriately. For instance, the FindBug was utilized for inspecting Java implementation, which also checked that present codes were safe from attacks.

VI. SECURITY ASSESSMENT AND SECURITY-RELATED TESTS

The SRP was tested to ascertain if it was secure and protected as it was intended. There many methodologies that can be used to test security related matter. Code average metric is common and popularly used metric when it comes to tests [7,57,58]. The entire source codes are supposed to be covered with different types of tests from unit tests to the last one which is the system test. Many tests as possible were supposed to be automated. The importance of security-related tests has become of great importance and in this study, they were not assumed. The security tests performed white-box and black-box penetration tests. If the software source code is not a public one, then it becomes known that the attacker is carrying out tests that are black-box [2,59]. In the case of white-box, the tester is aware and knows what the product being tested is. The importance of the penetration tests was to uncover new or previously known vulnerabilities that exist. Fuzz testing was also to be conducted; that is also a technique used in testing soft wares. Fuzz testing comprises of giving unexpected, random or invalid data straight to the inputs of particular software. The particular software is consequently tested for any failings or crashes of the code assertions that is build-in. The two types of fuzzing tests were utilized: mutation-based and generation-based. Fuzz testing is linked with the white-box and the black-box tests [8].

After the specification and implementation stage of the SRP, the following security assessments were tested and the methods of prevention designed appropriately as represented in the table below.

Potential Attack	Prevention Methods
Shared key	Dedicated key protects important data
Port scanning	Specific host and port for each process.
Sniffing	Configuration and encryption as per administrator requirements.
Unauthorized users	Addressed mechanisms of authorization and authentication.
Spoofing	Encryption of credentials
Replay attack	Unique call number replay
Poisoning of the local cache	Components responses and request were authenticated and encrypted
Service-Denial	Maximum load parameter addition in the configuration
Used algorithms that are weak	Permission was granted for more cryptographic algorithm.
Dictionary attack	Strong keys and passwords were forcefully generated for the protocol designer and implementation.

Table 1: Potential attacks and prevention methods

VII. PERFORMANCE RESULTS

The SRP was tested on a 2410M processor that is i5 regarding an Intel Core. The software was a VMware player. Each virtual machine had individual one core computer processing unit and a memory of 1GB. The operating system that was in use was named Fedora with a Core 15. The results obtained from the performance were in resemblance of those of remote method that was implemented with a mechanism of Java RMI [60,61]. The implementation of CORBA in Java also gives the same results just like this study [8,62]. The following table shows the result of SRP compared to that of CORBA and RMI.

Test	Requests per second
SRP flawless test with no hash	612
SRP flawless test that is MD5	513
SRP flawless text that is SHA1	498
SRP OFB or AES that is SHA1	502
RMI Java	399
CORBA Java	497

Table 2: SRP results compared to CORBA and RMI

VIII. CONCLUSION

According to the finding and the purpose of this study, it is possible to create a Secure Remote protocol. It can be defined with certain specification and implemented appropriately. The SRP should have a simple architecture and components to be able to address security issues more effectively and in a faster way. The SRP composed of an easy, new Secure Interface Description Language (SIDL) that is XML-based. The language is critical when it comes to remote protocols meant for the purpose of the interface definition. The SRP library is also present together with a translator of the SIDL. The quality of the present codes of SRP was assessed. Furthermore, the whole programming practices in use were secure, which means that the SRP implementation was also secure. It should be worth noting that some security testers were absent meaning that there are some doubts, on the whole, implementation process being secure. Security guidelines were also addressed in the study, with various security tests and assessments conducted. The assessment process was conducted, and the methods of prevention were also outlined properly. The performance results were also analysed and compared to those of Java CORBA and Java RMI.

As a result of this study, there are areas that have not been explored or addressed and further development is needed. Each and every day, new bugs and security issues arise. The security issues keep on changing as attackers adapt to the security tests and mechanisms that are present or have been created. The SRP is a very simple architecture and can be modified to suit certain requirements of clients [2,63,64]. Some of the security matters have not been well addressed. This leaves room for further development and innovation on security-related matters. Additionally, implementation by use of native languages may result in faster connections and network processes [7,65]. To conclude, this study shows that there is potential for creating the design, various specification and implementation of a universal, new network protocols intended for remote function, procedures or even methods.

REFERENCES

- [1.] Dong, L., & Chen, K. (2012). *Cryptographic protocol: Security analysis based on trusted freshness*. Berlin: Springer
- [2.] Medhi, D., & Ramasamy, K. (2007). *Network routing: Algorithms, protocols, and architectures*. Amsterdam: Elsevier/Morgan Kaufmann Publishers.
- [3.] Williams, A. (2001). *Java 2 network protocols black book*. Scottsdale, Ariz: Coriolis Group Books.
- [4.] Kizza, J. M. (2009). *A guide to computer network security*. London: Springer.
- [5.] Sutton, M., Greene, A., & Amini, P. (2007). *Fuzzing: Brute force vulnerability discovery*. Upper Saddle River, N.J: Addison-Wesley.
- [6.] Latze, C. (2010). *Towards a secure and user friendly authentication method for public wireless networks*. Berlin: Logos.
- [7.] Erickson, J. (2009). *Database technologies: Concepts, methodologies, tools, and applications*. Hershey, PA: Information Science Reference.
- [8.] Canetti, R. (2001, October). Universally composable security: A new paradigm for cryptographic protocols. In *Foundations of Computer Science, 2001. Proceedings. 42nd IEEE Symposium on* (pp. 136-145). IEEE.
- [9.] Abadi, M., & Gordon, A. D. (1997, April). A calculus for cryptographic protocols: The spi calculus. In *Proceedings of the 4th ACM conference on Computer and communications security* (pp. 36-47). ACM.
- [10.] Boneh, D., DeMillo, R. A., & Lipton, R. J. (1997, May). On the importance of checking cryptographic protocols for faults. In *Advances in Cryptology—EUROCRYPT'97* (pp. 37-51). Springer Berlin Heidelberg.
- [11.] Canetti, R. (2000). Security and composition of multiparty cryptographic protocols. *Journal of CRYPTOLOGY*, 13(1), 143-202.
- [12.] Abadi, M., & Gordon, A. D. (1998). A bisimulation method for cryptographic protocols. In *Programming Languages and Systems* (pp. 12-26). Springer Berlin Heidelberg.
- [13.] DeMillo, R. A., Lynch, N. A., & Merritt, M. J. (1982, May). Cryptographic protocols. In *Proceedings of the fourteenth annual ACM symposium on Theory of computing* (pp. 383-400). ACM.
- [14.] Mao, W. (2003). *Modern cryptography: theory and practice*. Prentice Hall Professional Technical Reference.
- [15.] Meadows, C. (1996). The NRL protocol analyzer: An overview. *The Journal of Logic Programming*, 26(2), 113-131.
- [16.] Mitchell, J. C., Ramanathan, A., Scedrov, A., & Teague, V. (2006). A probabilistic polynomial-time process calculus for the analysis of cryptographic protocols. *Theoretical Computer Science*, 353(1), 118-164.
- [17.] Comon, H., & Cortier, V. (2005). Tree automata with one memory set constraints and cryptographic protocols. *Theoretical Computer Science*, 331(1), 143-214.
- [18.] Amadio, R. M., Lugiez, D., & Vanackère, V. (2003). On the symbolic reduction of processes with cryptographic functions. *Theoretical Computer Science*, 290(1), 695-740.
- [19.] Comon-Lundh, H., & Cortier, V. (2003, June). New decidability results for fragments of first-order logic and application to cryptographic protocols. In *Rewriting Techniques and Applications* (pp. 148-164). Springer Berlin Heidelberg.
- [20.] Boyd, C. (1990). Hidden assumptions in cryptographic protocols. *IEE Proceedings E (Computers and Digital Techniques)*, 137(6), 433-436.
- [21.] Jorba, A. R., & Roca, J. C. (2007). *U.S. Patent No. 7,260,552*. Washington, DC: U.S. Patent and Trademark Office.
- [22.] Deng-Guo, Z. Y. B. F. (2006). Design and Analysis of Cryptographic Protocols for RFID [J]. *Chinese Journal of Computers*, 4, 008.
- [23.] Gordon, A. D., & Jeffrey, A. (2004). Types and effects for asymmetric cryptographic protocols. *Journal of Computer Security*, 12(3, 4), 435-483.
- [24.] Argyroudis, P. G., Verma, R., Tewari, H., & Mahony, D. O. (2004, August). Performance analysis of cryptographic protocols on handheld devices. In *Network Computing and Applications, 2004.(NCA 2004). Proceedings. Third IEEE International Symposium on* (pp. 169-174). IEEE.
- [25.] Fiore, M., & Abadi, M. (2001, June). Computing symbolic models for verifying cryptographic protocols. In *csfw* (p. 0160). IEEE.
- [26.] Maggi, P., & Sisto, R. (2002). Using SPIN to verify security properties of cryptographic protocols. In *Model Checking Software* (pp. 187-204). Springer Berlin Heidelberg.
- [27.] Blanchet, B., & Podelski, A. (2005). Verification of cryptographic protocols: tagging enforces termination. *Theoretical Computer Science*, 333(1), 67-90.
- [28.] Jorba, A. R., & Roca, J. C. (2007). *U.S. Patent No. 7,260,552*. Washington, DC: U.S. Patent and Trademark Office.
- [29.] Pass, R., & Rosen, A. (2008). New and improved constructions of nonmalleable cryptographic protocols. *SIAM Journal on Computing*, 38(2), 702-752.
- [30.] A. K. Dey, J. D. P. Rao, and T. D. Singh, Energy saving issue in Mobile Ad-hoc Networks. ISBN: 978-81-923288-1-2, 2013.
- [31.] J. D. P. Rao and R. K. Akuli, "A Brief Study on Measures to Improve Cyber Network Security," pp. 20–22, 2015.
- [32.] R. K. Akuli, J. D. P. Rao, and S. Kurariya, "A STUDY OF SECURITY MECHANISMS IMPLEMENTED IN NETWORK PROTOCOLS," Indian Streams Res. J. ISSN 2230-7850, vol. 5, no. 11, pp. 1–3, 2015.
- [33.] R. K. Akuli, J. D. P. Rao, and S. Kurariya, "NETWORK SECURITY MECHANISMS THROUGH OSI/ ISO NETWORK MODEL FOR UPPER LAYERS," Golden Res. Thoughts ISSN 2231-5063, vol. 5, no. 6, pp. 1–4, 2015.
- [34.] Taylor, D., Wu, T., Mavrogianopoulos, N., & Perrin, T. (2007). Using the Secure Remote Password (SRP) protocol for TLS authentication. *IETF Request for Comments*, 5054.
- [35.] Bheemarasetti, S., & Prathuri, C. (2002). *U.S. Patent Application No. 10/189,058*.
- [36.] Duncan, R. G., & Shabot, M. M. (2000). Secure remote access to a clinical data repository using a wireless personal digital assistant (PDA). In *Proceedings of the AMIA Symposium* (p. 210). American Medical Informatics Association.
- [37.] Kim, S. K., & Chung, M. G. (2009). More secure remote user authentication scheme. *Computer Communications*, 32(6), 1018-1021.
- [38.] Humenberger, M., Schraml, S., Sulzbachner, C., Belbachir, A. N., Srp, A., & Vajda, F. (2012, June). Embedded fall detection with a neural network and bio-inspired stereo vision. In *Computer Vision and Pattern Recognition Workshops (CVPRW), 2012 IEEE Computer Society Conference on* (pp. 60-67). IEEE.

- [39.] Abbott, B. P., Abbott, R., Adhikari, R., Ajith, P., Allen, B., Allen, G., ... & Araya, M. (2009). Search for gravitational-wave bursts in the first year of the fifth LIGO science run. *Physical Review D*, 80(10), 102001.
- [40.] Jahnke, J. H., D'entremont, M., & Stier, J. (2002). Facilitating the programming of the smart home. *Wireless Communications, IEEE*, 9(6), 70-76.
- [41.] Garavel, H., Lang, F., & Mateescu, R. (2001). An overview of CADP 2001.
- [42.] Baggen, R., Correia, J. P., Schill, K., & Visser, J. (2012). Standardized code quality benchmarking for improving software maintainability. *Software Quality Journal*, 20(2), 287-307.
- [43.] Boissinot, B., Darté, A., Rastello, F., De Dinechin, B. D., & Guillon, C. (2009, March). Revisiting out-of-SSA translation for correctness, code quality and efficiency. In *Proceedings of the 7th annual IEEE/ACM International Symposium on Code Generation and Optimization* (pp. 114-125). IEEE Computer Society.
- [44.] Lee, W. B., & Lee, C. D. (2008). A cryptographic key management solution for HIPAA privacy/security regulations. *Information Technology in Biomedicine, IEEE Transactions on*, 12(1), 34-41.
- [45.] Bishop, M. A. (2002). The art and science of computer security.
- [46.] Khalili, A., Katz, J., & Arbaugh, W. A. (2003, January). Toward secure key distribution in truly ad-hoc networks. In *Applications and the Internet Workshops, 2003. Proceedings. 2003 Symposium on* (pp. 342-346). IEEE.
- [47.] Glass, S., Hiller, T., Jacobs, S., & Perkins, C. (2000). *Mobile IP authentication, authorization, and accounting requirements* (No. RFC 2977).
- [48.] Aboba, B., & Wood, J. (2003). Authentication, authorization and accounting (AAA) transport profile.
- [49.] Karpati, D., Zilberman, A., Amos, E. B., & Halevi, I. (2006). *U.S. Patent Application No. 11/481,858*.
- [50.] Moustafa, H., Bourdon, G., & Gourhant, Y. (2006, September). Authentication, authorization and accounting (AAA) in hybrid ad hoc hotspot's environments. In *Proceedings of the 4th international workshop on Wireless mobile applications and services on WLAN hotspots* (pp. 37-46). ACM.
- [51.] Ferguson, P. (2000). Network ingress filtering: Defeating denial of service attacks which employ IP source address spoofing.
- [52.] Jung, J., Krishnamurthy, B., & Rabinovich, M. (2002, May). Flash crowds and denial of service attacks: Characterization and implications for CDNs and web sites. In *Proceedings of the 11th international conference on World Wide Web* (pp. 293-304). ACM.
- [53.] Kuzmanovic, A., & Knightly, E. W. (2003, August). Low-rate TCP-targeted denial of service attacks: the shrew vs. the mice and elephants. In *Proceedings of the 2003 conference on Applications, technologies, architectures, and protocols for computer communications* (pp. 75-86). ACM.
- [54.] Qin, F., Wang, C., Li, Z., Kim, H. S., Zhou, Y., & Wu, Y. (2006, December). Lift: A low-overhead practical information flow tracking system for detecting security attacks. In *Microarchitecture, 2006. MICRO-39. 39th Annual IEEE/ACM International Symposium on* (pp. 135-148). IEEE.
- [55.] Giffin, D. B., Levy, A., Stefan, D., Terei, D., Mazières, D., Mitchell, J. C., & Russo, A. (2012). Hails: Protecting data privacy in untrusted web applications. In *Presented as part of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)* (pp. 47-60).
- [56.] Shao, Z., Xue, C., Zhuge, Q., Qiu, M., Xiao, B., & Sha, E. H. M. (2006). Security protection and checking for embedded system integration against buffer overflow attacks via hardware/software. *Computers, IEEE Transactions on*, 55(4), 443-453.
- [57.] Subashini, S., & Kavitha, V. (2011). A survey on security issues in service delivery models of cloud computing. *Journal of network and computer applications*, 34(1), 1-11.
- [58.] Curphey, M., & Arawo, R. (2006). Web application security assessment tools. *Security & Privacy, IEEE*, 4(4), 32-41.
- [59.] Chellappa, R. K., & Pavlou, P. A. (2002). Perceived information security, financial liability and consumer trust in electronic commerce transactions. *Logistics Information Management*, 15(5/6), 358-368.
- [60.] Bennani, T., Blain, L., Courtes, L., Fabre, J. C., Killijian, M. O., Marsden, E., & Taiani, F. (2004, June). Implementing simple replication protocols using corba portable interceptors and java serialization. In *Dependable Systems and Networks, 2004 International Conference on* (pp. 549-554). IEEE.
- [61.] Siegel, J. (2000). *CORBA 3 fundamentals and programming* (Vol. 2). New York, NY, USA:: John Wiley & Sons.
- [62.] Gray, N. A. (2004, April). Comparison of Web Services, Java-RMI, and CORBA service implementations. In *Proceedings of the 5th Australasian Workshop on Software and System Architectures at ASWEC* (pp. 52-63).
- [63.] Lee, W. J., Shin, Y., Lee, J., Kim, J. W., Nah, J. H., Jung, S., ... & Han, T. D. (2013, July). SGRT: A mobile GPU architecture for real-time ray tracing. In *Proceedings of the 5th high-performance graphics conference* (pp. 109-119). ACM.
- [64.] Blodget, B., James-Roxby, P., Keller, E., McMillan, S., & Sundararajan, P. (2003). A self-reconfiguring platform. In *Field Programmable Logic and Application* (pp. 565-574). Springer Berlin Heidelberg.
- [65.] Almeida, N., Busson, P., Faure, J. L., Gachelin, O., Gras, P., Mandjavidze, I., ... & Varela, J. (2005). The selective read-out processor for the CMS electromagnetic calorimeter. *Nuclear Science, IEEE Transactions on*, 52(3), 772-777.
- [66.] Topuz, B. (2010). Development of a static analysis tool to find security vulnerabilities in java applications.
- [67.] Aner, Y. (2009). *Securing the Sage Notebook* (Doctoral dissertation, Master's thesis, Royal Holloway, University of London).